SOLAR▸SWARM

Change Management
Martin Pham | Perth, AU

Solar Swarm — Rollback Procedures

Use when a deployment causes a P1 or P2 incident.

Quick Reference

SCENARIO	ACTION	TIME TO RECOVER
Bad code deployed	`git revert` + redeploy	5–10 min
Database migration broke schema	Restore from backup	10–15 min
Wrong `.env` config	Edit `.env` + restart	2 min
Scheduler job looping	Kill job, fix code	5 min
CRM sync corrupted cache	Clear `crm_cache` table	2 min
AI prompt regression	Revert prompt in git	5 min

Procedure 1 — Code Rollback

Use when the deployed code causes errors but the database is intact.

```
# 1. Find the last working commit
git log --oneline -10

# 2. Revert to previous commit (creates a new revert commit)
git revert HEAD --no-edit

# 3. Restart services
pkill -f "python main.py"
python main.py &

# 4. Verify health
curl http://localhost:5003/api/health
curl http://localhost:5000/health
```

If the revert commit approach is too slow, hard reset to previous:

```
# WARNING: This discards the bad commit permanently
git reset --hard HEAD~1
python main.py &
```

Procedure 2 — Database Rollback

Use when a schema migration corrupted the database.

Before any schema migration — always backup first:

```
cp swarm.db swarm.db.backup-$(date +%Y%m%d-%H%M)
```

Restore from backup:

```
# 1. Stop services
pkill -f "python main.py"

# 2. Restore backup
cp swarm.db.backup-YYYYMMDD-HHMM swarm.db

# 3. Verify database
sqlite3 swarm.db ".tables"

# 4. Restart
python main.py &
```

If no backup exists (emergency):

```
# Recreate database from schema (DATA WILL BE LOST — last resort only)
rm swarm.db
python -c "from memory.database import init_db; init_db()"
python main.py &
```

Procedure 3 — Configuration Rollback

Use when a `.env` change caused a misconfiguration.

```
# 1. Open .env and restore previous value
nano .env

# 2. Restart services (no redeploy needed)
pkill -f "python main.py"
python main.py &

# 3. Verify
python cli.py status
```

Procedure 4 — CRM Cache Corruption

Use when CRM sync wrote bad data to `crm_cache`.

```
# 1. Clear the cache table
sqlite3 swarm.db "DELETE FROM crm_cache;"

# 2. Force an immediate CRM sync
python -c "from api.crm_sync import run; run()"

# 3. Verify cache repopulated
sqlite3 swarm.db "SELECT cache_key, cached_at FROM crm_cache LIMIT 10;"
```

Procedure 5 — Scheduler Emergency Stop

Use when a scheduler job is looping, consuming API budget, or stuck.

```
# 1. Kill the main process
pkill -f "python main.py"

# 2. Identify the problematic job in main.py
# Comment out the scheduler.add_job() call for that job

# 3. Restart with the job disabled
python main.py &

# 4. Fix the job code, then re-enable
```

Procedure 6 — Circuit Breaker Reset (Stuck at Red)

Use when the circuit breaker halted the swarm and you've fixed the underlying issue.

```
# Via API (requires GATE_API_KEY)
curl -X POST http://localhost:5000/approve-breaker \
  -H "Authorization: Bearer YOUR_GATE_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"approved_by": "martin"}'

# Direct database reset (if API is down)
sqlite3 swarm.db "INSERT INTO circuit_breaker_log (level, reason, created_at) VALUES ('green', 'M
```

Procedure 7 — AI Voice Call Emergency Disable

Use when AI calls are misfiring or calling wrong numbers.

```
# 1. Set RETELL_API_KEY to empty in .env (disables all outbound calls)
nano .env
# Set: RETELL_API_KEY=

# 2. Restart services
pkill -f "python main.py"
python main.py &

# 3. Verify voice status
curl http://localhost:5003/api/voice/status
# Should return: {"status": "offline"}
```

Post-Rollback Checklist

After any rollback, complete these steps before considering the incident closed:

- [ ] Root cause identified (write 1-2 sentences)
- [ ] All services health-checked and green
- [ ] Test lead processed end-to-end
- [ ] Client notified if they were affected (use plain English, no jargon)
- [ ] Post-mortem added to incident log
- [ ] Preventative measure identified (code fix, test, or process change)
- [ ] Monitoring added to detect the same issue earlier next time

---

Incident Log Template

```
Date:           YYYY-MM-DD HH:MM AWST
Duration:       X minutes
Severity:       P1 / P2 / P3
Trigger:        What caused the incident
Impact:         What stopped working, which clients affected
Detection:      How it was found (alert / client report / monitoring)
Resolution:     Which procedure was used
Root cause:     One sentence explaining why it happened
Prevention:     What change prevents recurrence
```